COMP3320/6464 Week 3, Lecture 2

Rhys Hawkins

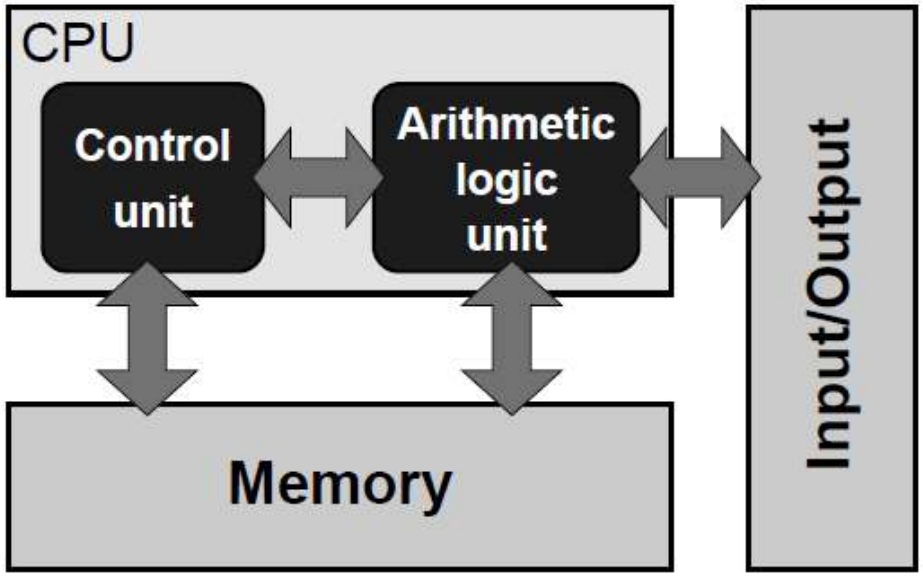
Semester 2, 2024

Mid-Semester Test

- August 13th, 2pm, Fulton Muir Rm 2.02
- We will start at approximately 2:10pm
- 15 Minutes Reading Time + 60 Minutes
- Must bring student ID card.
- Open Book Test
- 3 Sections with 20 Marks each
  - Floating point representation and precision
  - CPU Architecture (Cache, Pipelines, Super-Scalar)
  - Optimization
- Students with EAPs will receive an email from me shortly with separate instructions. Please respond to this email.
- If you can't make the Mid-Semester for a valid reason (illness, caring responsibilities etc), get in touch as soon as you can.

Cache-based microprocessor

Figure 1.1: Stored-program computer architectural concept. The "program," which feeds the control unit, is stored in memory together with any data the arithmetic unit requires.



Data-centric memory hierarchy

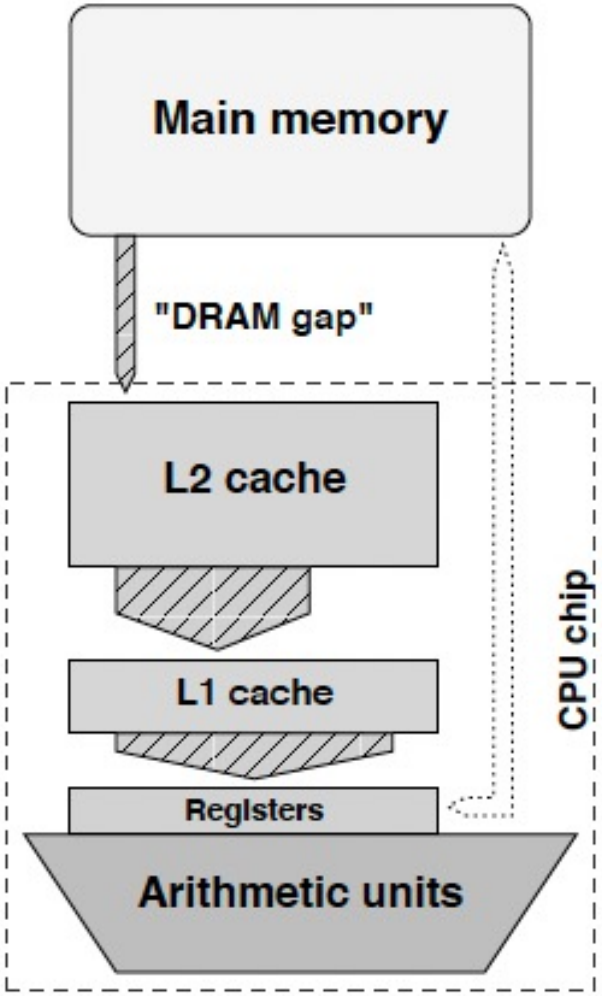
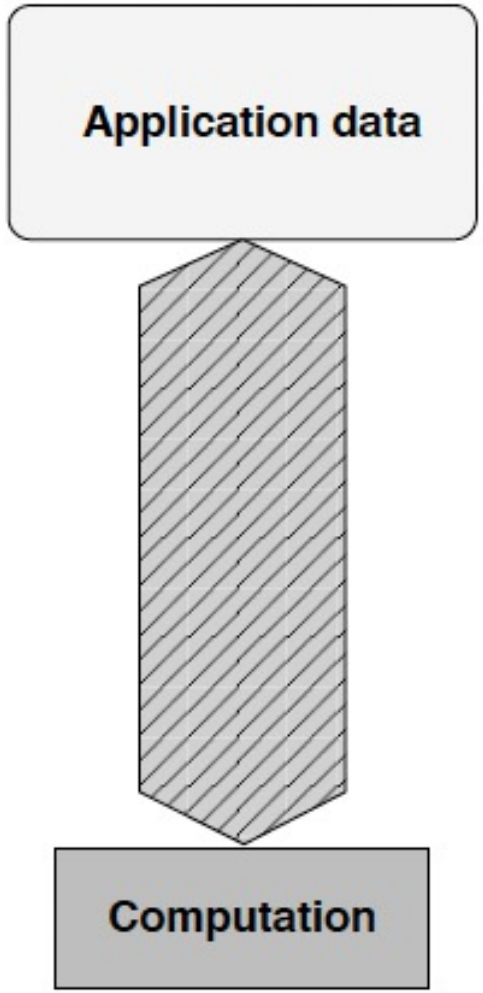


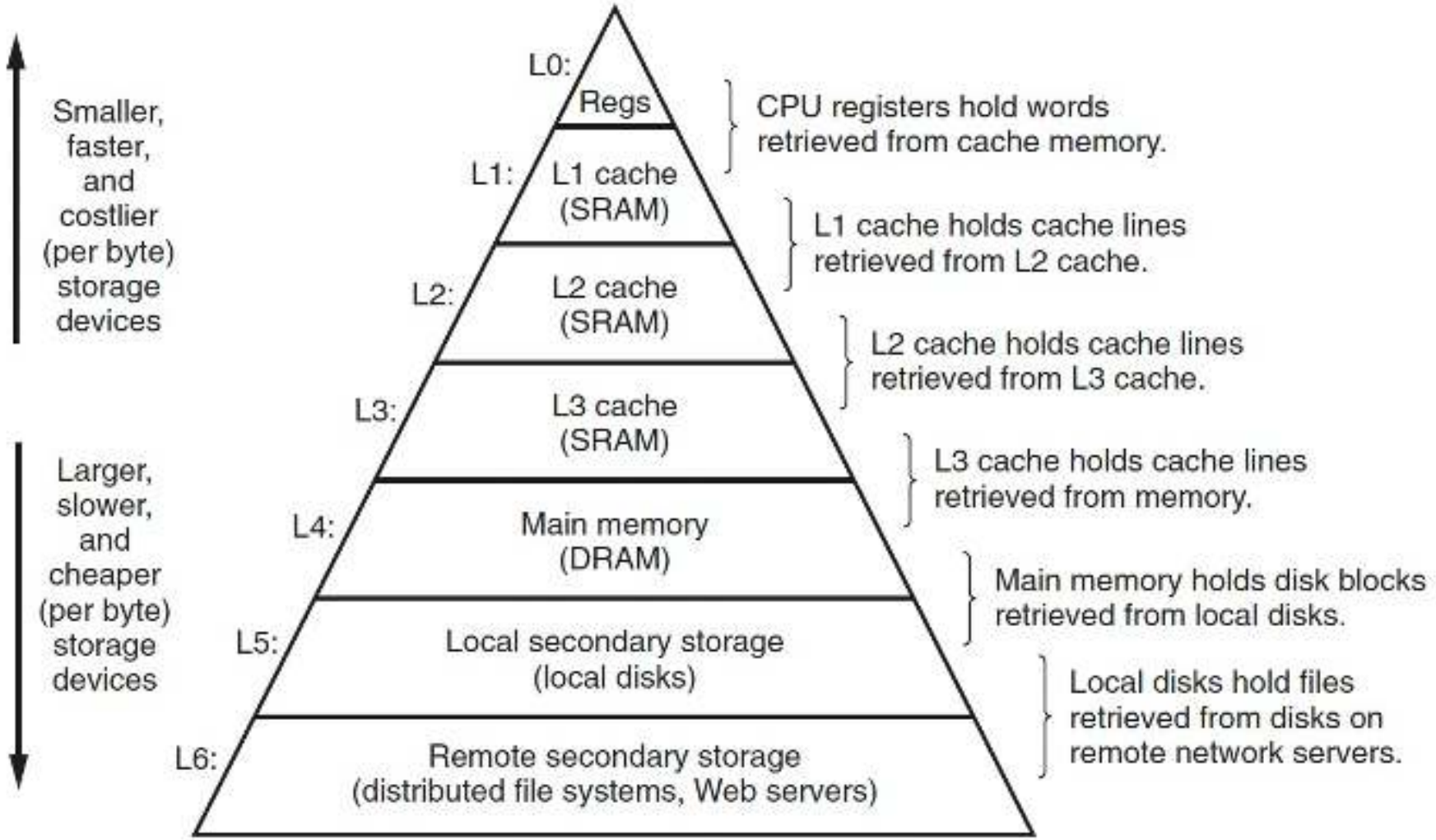
Figure 1.3: (Left) Simplified data-centric memory hierarchy in a cache-based microprocessor (direct access paths from registers to memory are not available on all architectures). There is usually a separate L1 cache for instructions. (Right) The "DRAM gap" denotes the large discrepancy between main memory and cache bandwidths. This model must be mapped to the data access requirements of an application.



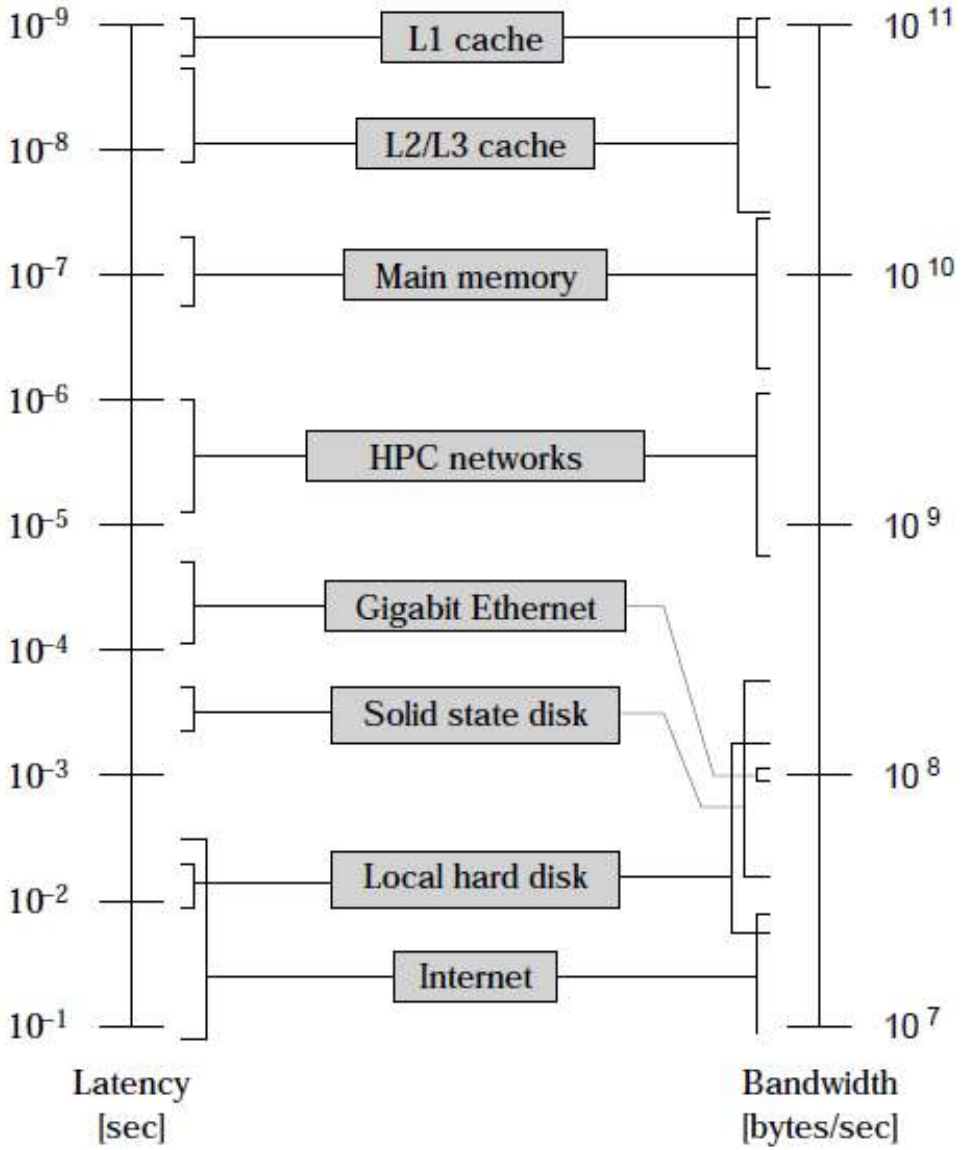
Memory Hierarchy

The central idea of a memory hierarchy is that for each k, the faster and smaller storage device at level k serves as a cache for the larger and slower storage device at level $k + 1$

Memory Hierarchy



Memory Hierarchy



Intel Cascade Lake (GADI)

- L1 Cache 4 Cycles
- L2 Cache 14 Cycles
- L3 Cache 50-70 Cycles
- DRAM $\sim$ 200 Cycles

For comparison

- Floating point add 3 Cycles
- Floating point multiply 5 Cycles

Memory Locality

Temporal Locality – Close in time

- Programs that repeatedly reference the same variables enjoy good temporal locality.

Spatial Locality – Close in space

- For programs with stride-k reference patterns, the smaller the stride the better the spatial locality.

Loops have good temporal and spatial locality with respect to instruction fetches.

- The smaller the loop body and the greater the number of loop iterations, the better the locality.

Good Locality Example 1.

```
1  int sumvec(int v[N])
2  {
3      int i, sum = 0;
4
5      for (i = 0; i < N; i++)
6          sum += v[i];
7      return sum;
8  }
```

Address	0	4	8	12	16	20	24	28
Contents	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Access order	1	2	3	4	5	6	7	8

Good Locality Example 2.

```
1  int sumarrayrows(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (i = 0; i < M; i++)
6          for (j = 0; j < N; j++)
7              sum += a[i][j];
8      return sum;
9  }
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	2	3	4	5	6

Operations of CPU Cache

What happens when we do a load

- Check L1, L2, L3 in order
- If it is in none, then read from main memory
- Store in each level of cache (what happens to other stored cache lines?)

Replacement Policies

- **Read** least-recent-used (LRU) most common

E.g. GADI CPUs have 32kB L1-cache arranged in 512 blocks of 64 bytes

Operations of CPU Cache : Write

What happens when a write causes a cache hit?

- Write-through
  - Immediate write to main memory (slow)
- Write-back
  - Written to cache
  - Cache line marked as “dirty”
  - Written back to memory when cache line is evicted

Operations of CPU Cache : Write

What happens when a write causes a cache miss?

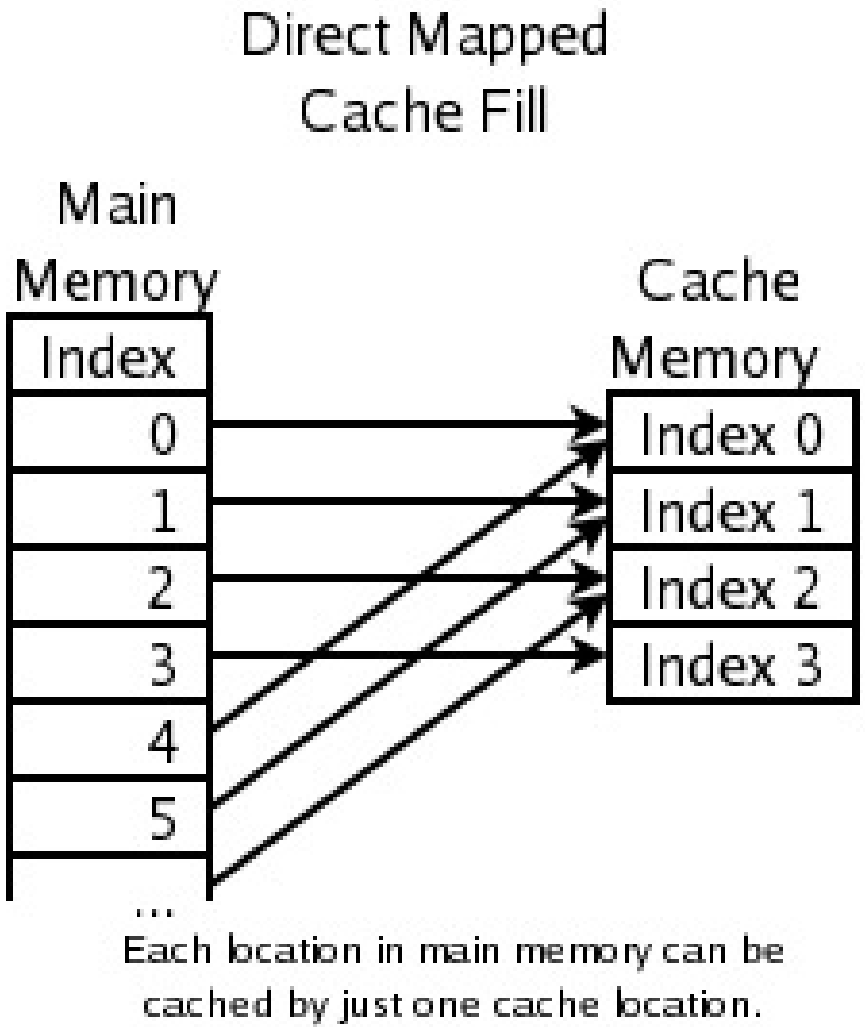
- Write allocate
  - For cache consistency, the cache line is read into cache
  - Write data to cache
  - Written back to memory when cache line is evicted
- Write-through
  - Direct write to memory

Operations of CPU Cache : Read vs Write

- As a general rule, cache misses for write are more expensive than cache misses for read
- In code where there is a trade off between read stride and write stride, prefer lower write stride.
- Examples of this next week

Cache Associativity - Direct Mapping

How memory is stored in cache - Direct Mapping



Downside of direct mapping

Cache Thrashing

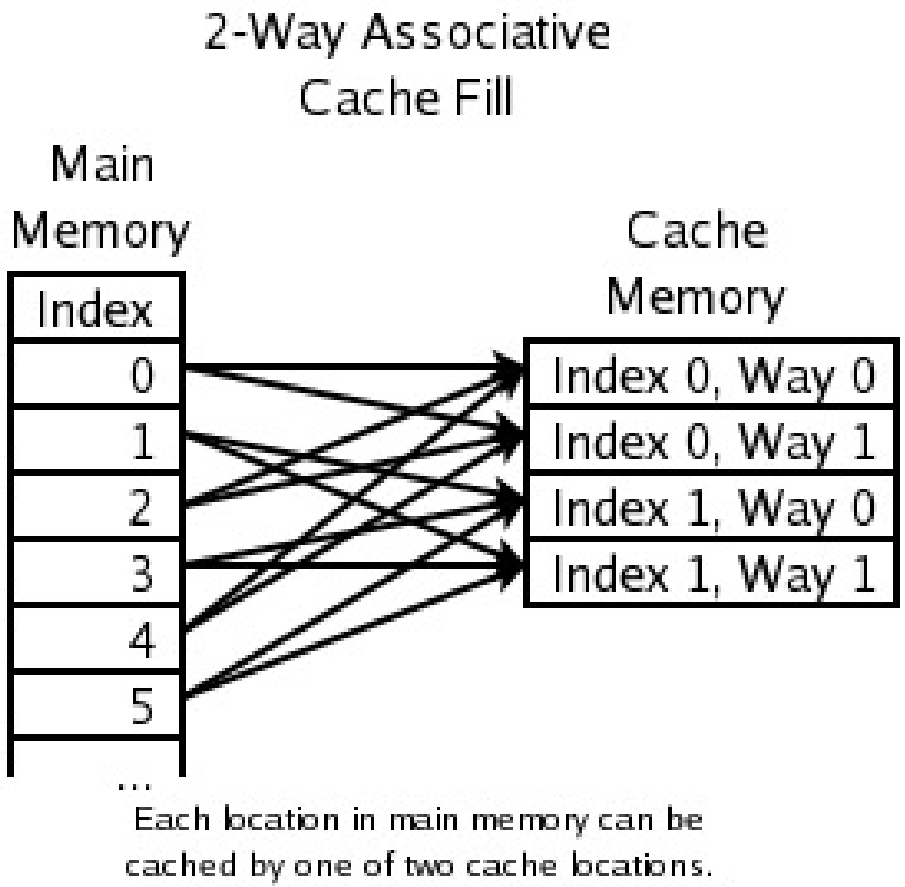
```
1 do i=1,N,CACHE_SIZE_IN_BYTES/8
2    A(i) = B(i) + C(i) * D(i)
3 enddo
```

Repeated access of same cache element from different memory is slow

- Guaranteed repeated cache miss
- Memory cycle time $>$ memory access time

Cache Associativity - N-way associative

How memory is stored in cache - N-way associative



Cache Friendly Code

Make the common case go fast.

- Programs often spend most of their time in a few core functions.
- These functions often spend most of their time in a few loops.
- Focus on the inner loops of the core functions and ignore the rest.

Minimise the number of cache misses in each inner loop.

- Loops with better miss rates will run faster
  - all other things being equal (such as the total number of loads and stores)
- Cache has a block size of B bytes then a stride-k reference pattern (where k is expressed in words) results in an average of:

$$\min\left(1, \frac{\text{wordsize} \times k}{B}\right) \text{ misses per loop iteration}$$

Cache Friendly Code – Example 1

Sum Vector

```
int sumvec(int v[N])
{
    int i, sum = 0;

    for (i = 0; i < N; i++)
        sum += v[i];
    return sum;
}
```

Address	0	4	8	12	16	20	24	28
Contents	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Access order	1	2	3	4	5	6	7	8

- Stride = 1
- Words are 4 bytes (Integers)
- Cache blocks are 4 words
- Cache is initially empty (a cold cache).

Then the pattern of hits and misses for references to v

v[i]	$i=0$	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$
Access order, [h]it or [m]iss	1 **[m]**	2 [h]	3 [h]	4 [h]	5 **[m]**	6 [h]	7 [h]	8 [h]

Local variables – good temporal locality

- Any reasonable optimising compiler will use registers for local variables, the highest level of the memory hierarchy.

Stride-1 reference patterns - good spatial locality

- Caches at all levels of the memory hierarchy store data as contiguous blocks

Cache Friendly Code – Example 2

Sum Array

```
for (i = 0; i < N; i ++) {
  for (j = 0; j < N; j ++) {
    s = s + A[i][j];
  }
}
```

- Stride = 1
- Words are 4 bytes
- Cache blocks are 4 words
- Cache is initially empty (a cold cache).

Then the pattern of hits and misses for references to A

a[i][j]	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
i = 0	1 **[m]**	2 [h]	3 [h]	4 [h]	5 **[m]**	6 [h]	7 [h]	8 [h]
i = 1	9 **[m]**	10 [h]	11 [h]	12 [h]	13 **[m]**	14 [h]	15 [h]	16 [h]
i = 2	17 **[m]**	18 [h]	19 [h]	20 [h]	21 **[m]**	22 [h]	23 [h]	24 [h]
i = 3	25 **[m]**	26 [h]	27 [h]	28 [h]	29 **[m]**	30 [h]	31 [h]	32 [h]

Cache Unfriendly Code – Example 3

Sum Array

```
for (j = 0; j < N; j ++) {
  for (i = 0; i < N; i ++) {
    s = s + A[i][j];
  }
}
```

- Stride = 1
- Words are 4 bytes
- Cache blocks are 4 words
- Cache is initially empty (a cold cache).

Then the pattern of hits (0) and misses for references to A

a[i][j]	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
i = 0	1 [m]	5 [m]	9 [m]	13 [m]	17 [m]	21 [m]	25 [m]	29 [m]
i = 1	2 [m]	6 [m]	10 [m]	14 [m]	18 [m]	22 [m]	26 [m]	30 [m]
i = 2	3 [m]	7 [m]	11 [m]	15 [m]	19 [m]	23 [m]	27 [m]	31 [m]
i = 3	4 [m]	8 [m]	12 [m]	16 [m]	20 [m]	24 [m]	28 [m]	32 [m]

Cache Performance : Real World Measurement

```
void matrix_add_friendly(int N, int M,
                         const float *A,
                         const float *B,
                         float *C)
{
  int i, j;
  for (i = 0; i < N; i ++) {
    for (j = 0; j < M; j ++) {
      C[M*i + j] = A[M*i + j] + B[M*i + j];
    }
  }
}
```

- `C[M*i + j]` $\equiv$ `C[i][j]`
- Stride = 1
- 2 Loads, 1 add, 1 Store per iteration
- Cache Line Size 64 Bytes, 16 floats (4 bytes) per cache line
- Expect Cache Miss every 16 iterations of `j`

Cache Performance : Real World Measurement

```
void matrix_add_unfriendly(int N, int M,
                           const float *A,
                           const float *B,
                           float *C)
{
  int i, j;
  for (j = 0; j < M; j ++) {
    for (i = 0; i < N; i ++) {
      C[M*i + j] = A[M*i + j] + B[M*i + j];
    }
  }
}
```

- `C[M*i + j]` $\equiv$ `C[i][j]`
- Stride = M
- 2 Loads, 1 add, 1 Store per iteration
- Cache Line Size 64 Bytes, 16 floats (4 bytes) per cache line
- If M $\geq$ 16, expect cache miss every iteration!

Cache Performance : Real World Measurement

- With M = 16
- N sufficiently large to flush L1 cache size
- 1000 repetitions
- Using PAPI to count L1 Cache misses

	Friendly	Unfriendly
Float Read/Writes	983040	983040
L1 Cache Misses	61677	983594
Time (μs)	2153	3623

- Note 983040/61677 $\approx$ 16
- Optimal is `64/sizeof(float)` = 16

Low Level Benchmarking

- A program that tests some specific feature of the architecture e.g., peak performance or memory bandwidth
- Useful for approximate performance prediction
- Cannot predict the behaviour of real application code

STREAM benchmarks

type	kernel	DP words	flops	B_c
COPY	`A(:)=B(:)`	2 (3)	0	N/A
SCALE	`A(:)=s*B(:)`	2 (3)	1	2.0 (3.0)
ADD	`A(:)=B(:)+C(:)`	3 (4)	1	3.0 (4.0)
TRIAD	`A(:)=B(:)+s*C(:)`	3 (4)	2	1.5 (2.0)

Table 3.2: The STREAM benchmark kernels with their respective data transfer volumes (third column) and floating-point operations (fourth column) per iteration. Numbers in brackets take write allocates into account.

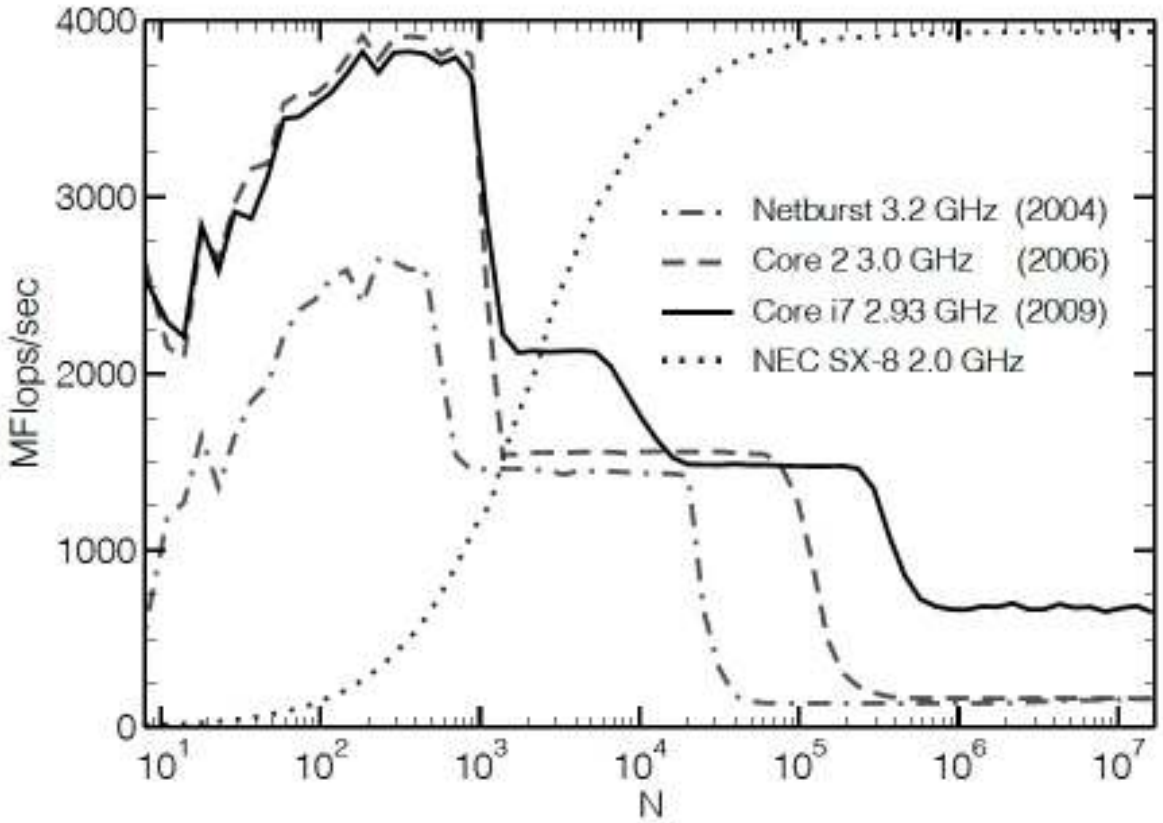
STREAM : Vector Triad Benchmark

```
for (i = 0; i < N; i ++) {
  A[i] = B[i] + C[i]*D[i];
}
```

- Depending on *N*, might run very fast and need to be run multiple times.
- For memory speed testing we have to make sure that the arrays are larger than cache sizes

Vector Triad Benchmark

Figure 1.4: Serial vector triad performance versus loop length for several generations of Intel processor architectures (clock speed and year of introduction is indicated), and the NEC SX-8 vector processor. Note the entirely different performance characteristics of the latter.



Structure of Arrays vs Array of Structures

- Array of Structures

```
struct point {
  float x, y, z;
};
struct point particle[N];

/* Loop 1 */
for (i = 0; i < N; i ++) {
  r = sqrt(particle[i].x*particle[i].x +
           particle[i].y*particle[i].y +
           particle[i].z*particle[i].z);
}

/* Loop 2 */
mean_x = 0.0
for (i = 0; i < N; i ++) {
  mean_x = mean_x + particle[i].x;
}
mean_x = mean_x/N;
```

- 1st Loop: all parts of structure used so cache use is good
- 2nd Loop: only using x, so the stride is effectively 3.

Structure of Arrays vs Array of Structures

- Structure of Arrays

```
struct particlelist {
  float x[N], y[N], z[N];
};
struct particlelist particle;

/* Loop 1 */
for (i = 0; i < N; i ++) {
  r = sqrt(particle.x[i]*particle.x[i] +
           particle.y[i]*particle.y[i] +
           particle.z[i]*particle.z[i]);
}

/* Loop 2 */
mean_x = 0.0
for (i = 0; i < N; i ++) {
  mean_x = mean_x + particle.x[i];
}
mean_x = mean_x/N;
```

- 1st Loop: For N $>$ 16, each initial reference to x, y, z causes a cache miss
- 2nd Loop: only using x, but stride is still 1

Structure of Arrays vs Array of Structures : Summary

- For first loop, and N iterations
  - AofS: Struct 3 $\times$ 4 bytes, meaning cache miss every 12/64 iteratons
  - SofA: 3 cache misses every 16 iterations
  - Overall, for large N, SofA and AofS have the same number of cache misses for loop 1!
- For second loop, SofA has factor of 3 less cache misses.
- Design your data around your use cases.
- There are implications for SIMD instructions we'll cover next week.
- Do not fill up structures with extra information not relevant to computation, e.g. do not do this:

```
struct point {
  float x, y, z;
  char particle_name[256];
};
struct point particle[N];
```

Summary

- Temporal locality: use the same memory repeatedly (e.g. accumulator variables)
- Spatial locality: minimise stride
- Design structures with computation in mind
- Cache Prefetching
  - Not covered, but there are commands to prefetch memory into cache that can be used to improve performance
  - CPUs do this automatically if memory accesses are predictable/regular